

UE19CS251

DESIGN AND ANALYSIS OF ALGORITHMS

UNIT 5: Limitations of Algorithmic Power and
Coping with the Limitations

Dynamic Programming

PES University

Outline

Concepts covered

- Dynamic Programming
 - Introduction
 - Fibonacci numbers
 - Binomial Coefficients

1 Introduction

Dynamic Programming is a general algorithm design technique for solving problems defined by recurrences with overlapping subproblems

- Invented by American mathematician Richard Bellman in the 1950s to solve optimization problems and later assimilated by CS
- “Programming” here means “planning”
- Main idea:
 - set up a recurrence relating a solution to a larger instance to solutions of some smaller instances
 - solve smaller instances once
 - record solutions in a table
 - extract solution to the initial instance from that table

2 Example: Fibonacci Numbers

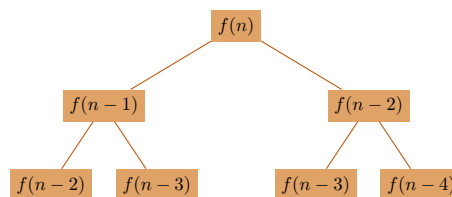
- Recall definition of Fibonacci numbers:

$$f(n) = f(n-1) + f(n-2)$$

$$f(0) = 0$$

$$f(1) = 1$$

- <2-> Computing the n^{th} Fibonacci number recursively (top-down):



3 Example: Fibonacci Numbers

Computing the n^{th} Fibonacci number using bottom-up iteration and recording results:

$$f(0) = 0$$

$$f(1) = 1$$

$$f(2) = 0 + 1 = 1$$

$$f(3) = 1 + 1 = 2$$

$$f(4) = 1 + 2 = 3$$

$\vdots$

Efficiency:

- time: $\Theta(n)$
- space: $\Theta(n)$ or $\Theta(1)$

4 Algorithm Examples

- Computing a binomial coefficient
- Warshall's algorithm for transitive closure
- Floyd's algorithm for all-pairs shortest paths
- Constructing an optimal binary search tree
- Some instances of difficult discrete optimization problems:
 - traveling salesman
 - knapsack

5 Binomial Coefficient

- Binomial coefficients are coefficients of the binomial formula:

$$(a + b)^n = C(n, 0)a^n b^0 + \dots + C(n, k)a^{n-k}b^k + \dots + C(n, n)a^0 b^n$$

- <2-> Recurrence:

$$C(n, k) = C(n-1, k) + C(n-1, k-1) \quad \text{for } n > k > 0$$

$$C(n, 0) = 1, C(n, n) = 1 \quad \text{for } n \geq 0$$

- <3-> Value of $C(n, k)$ can be computed by filling a table:

	0	1	2	.	.	.	k-1	k
0	1							
1	1	1						
.								
.								
.								
n-1							$C(n-1, k-1)$	$C(n-1, k)$
n								$C(n, k)$

6 Binomial Coefficient Algorithm

Dynamic Programming Binomial Coefficient Algorithm

```

1: procedure BINOMIAL( $n, k$ )
2:   ▷ Input: Integers  $n \geq 0, k \geq 0$ 
3:   ▷ Output:  $C(n, k)$ 
4:   for  $i \leftarrow 0$  to  $n$  do
5:     for  $j \leftarrow 0$  to  $\min(i, k)$  do
6:       if  $j=0$  or  $j=i$  then
7:          $C(i, j) \leftarrow 1$ 
8:       else  $C[i, j] = C[i-1, j] + C[i-1, j-1]$ 
9:   return  $C[n, k]$ 

```

- <2-> Time: $\Theta(nk)$
- <2-> Space: $\Theta(nk)$

7 Think About It

- What does dynamic programming have in common with divide-and-conquer? What is a principal difference between them?
- <2-> The coin change problem does not have an optimal greedy solution in all cases (ex: coins 1,20,25 and amount 40). Is there a dynamic programming based algorithm that can solve all cases of the coin change problem?